



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Inteligencia Artificial. Grupo 7

Profesor: Dr. Jose Abel Herrera Camacho

PROYECTO

Uniform Cost Search

Apellido paterno	Apellido materno	Nombre(s)
Balam	Flores	Gabriel Patricio

Abril 2025

Índice

1. Descripción	3
2. Requisitos del sistema	3
3. Cómo ejecutar el programa	3
4. Ejemplo de uso	4
5. Pseudocódigo del algoritmo Uniform Cost Search	7
6. Formato del archivo JSON de entrada	7
7. Estructura del proyecto	8

1. Descripción

El proyecto tiene como objetivo la visualización interactiva del algoritmo de búsqueda de costo uniforme (Uniform Cost Search, UCS) aplicado sobre un grafo definido por el usuario.

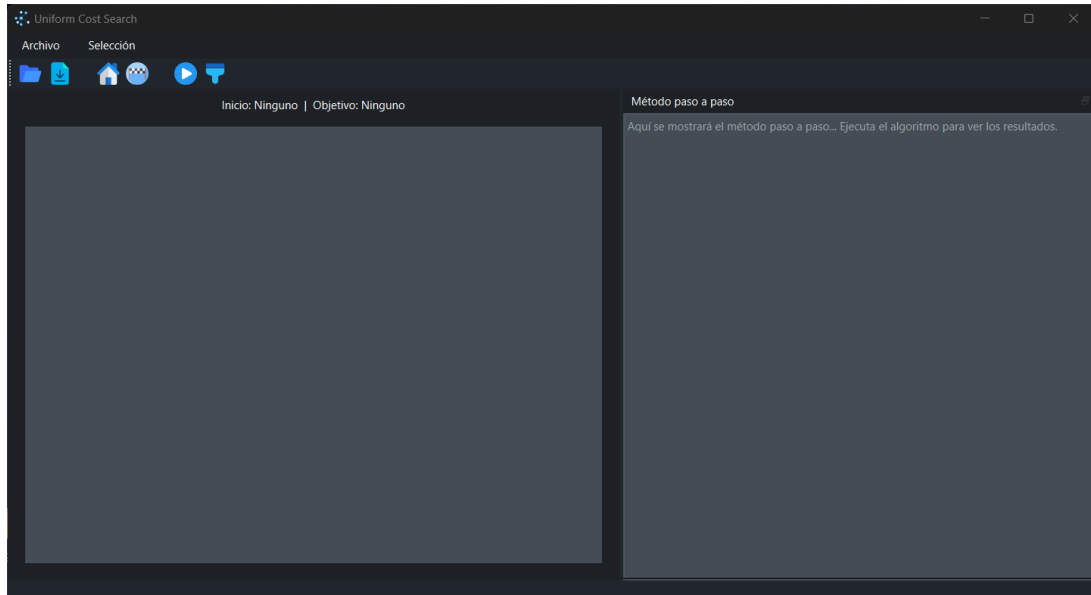


Figura 1: Interfaz de usuario

2. Requisitos del sistema

- Python 3.10 o superior
- PySide6
- NetworkX
- pyqtgraph

3. Cómo ejecutar el programa

Para ejecutar el programa desde la terminal, navega hasta la carpeta `Proy` y ejecuta el archivo `main.py` con el siguiente comando:

```
py main.py
```

También es posible iniciar el programa mediante un acceso directo. En la ruta `/output/main` se encuentra el archivo ejecutable `main.exe`, el cual puede abrirse directamente haciendo doble clic.

4. Ejemplo de uso

En la parte superior izquierda de la interfaz se encuentran las barras de menú y de herramientas, las cuales permiten acceder a las principales funciones del programa.

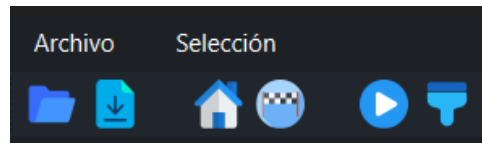


Figura 2: Menú y barra de herramientas

El menú **Archivo** incluye las siguientes opciones:

- **Abrir:** Permite seleccionar un archivo JSON para cargar el grafo.
- **Descargar:** Guarda el contenido actual de la caja de texto en un archivo.
- **Acerca de:** Abre el repositorio del proyecto en GitHub.
- **Salir:** Cierra la aplicación.

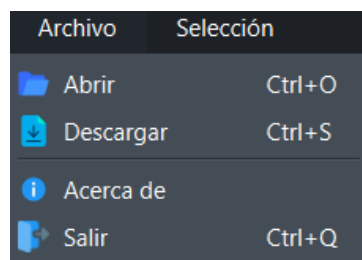


Figura 3: Opciones del menú Archivo

El menú **Selección** permite definir los nodos clave del grafo para ejecutar el algoritmo de búsqueda. Incluye las siguientes opciones:

- **Nodo de inicio:** Permite seleccionar el nodo desde el cual comenzará la búsqueda.
- **Nodo objetivo:** Permite seleccionar el nodo al que se desea llegar.

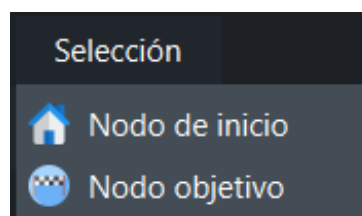


Figura 4: Opciones del menú Selección

Barra de herramientas:

Imagen	Descripción de la herramienta
	Permite seleccionar un archivo JSON para cargar el grafo en la interfaz.
	Guarda el contenido de la caja de texto en un archivo.
	Permite seleccionar el nodo desde el cual comenzará la búsqueda.
	Permite seleccionar el nodo al que se desea llegar.
	Ejecuta el algoritmo de búsqueda.
	Reinicia la ruta y el cuadro de texto.

Cuadro 1: Herramientas de la barra superior

Una vez seleccionado el archivo JSON, el grafo se muestra en el área central de la interfaz. El usuario puede definir los nodos de inicio y objetivo haciendo clic directamente sobre los nodos deseados en el grafo.

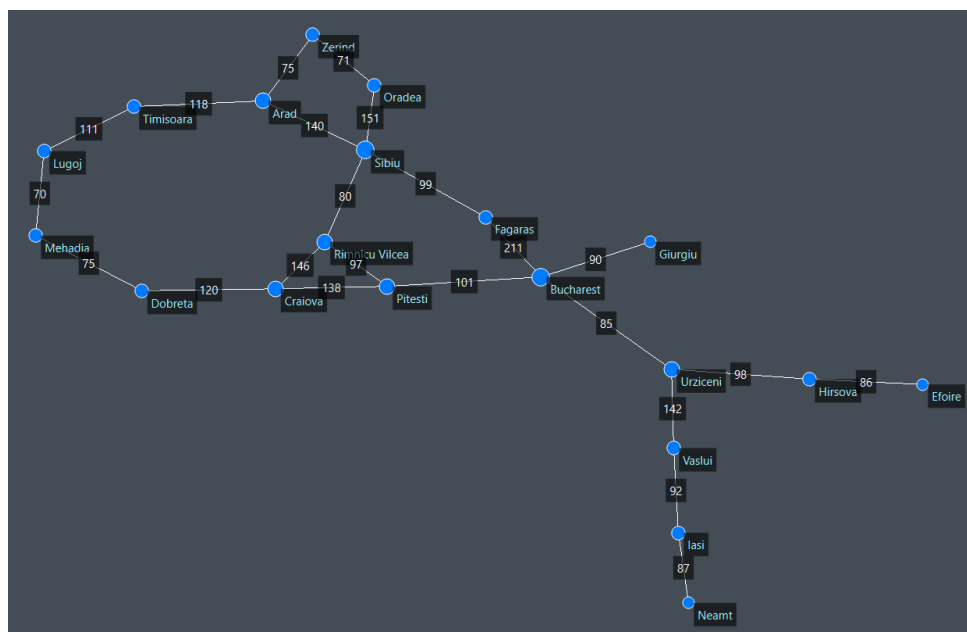


Figura 5: Ejemplo de grafo

Ejecución del algoritmo: Una vez seleccionados los nodos, el algoritmo se puede ejecutar utilizando la barra de herramientas o presionando **ctrl + Enter**. Al ejecutar el algoritmo, el área del grafo mostrará la ruta óptima coloreada, y el cuadro de texto mostrará el paso a paso del proceso.

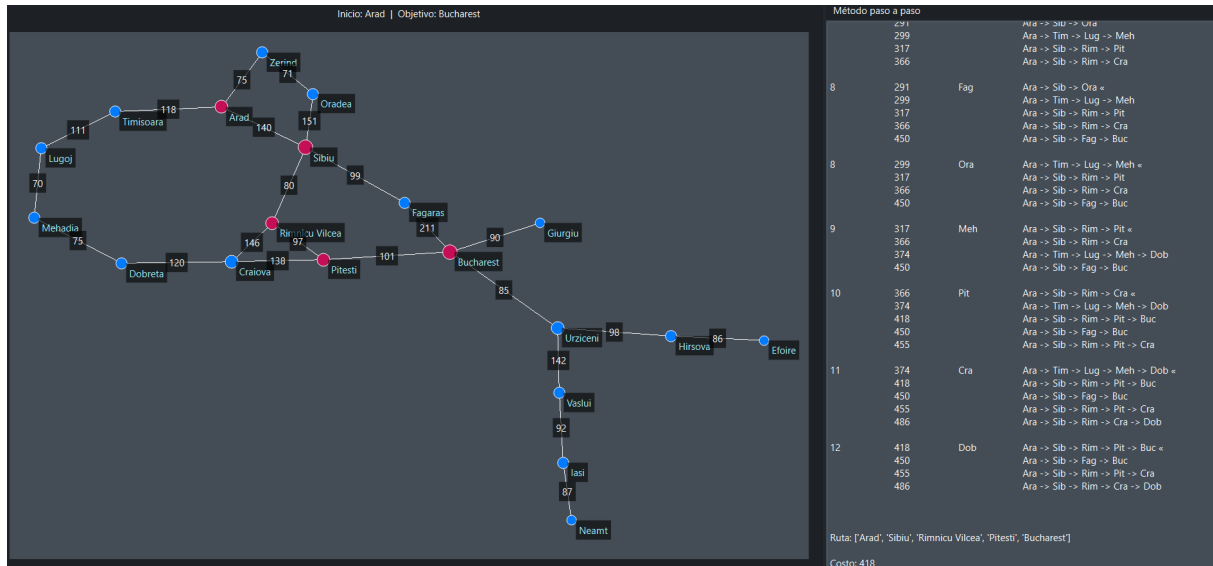


Figura 6: Ejemplo de Arad a Bucharest

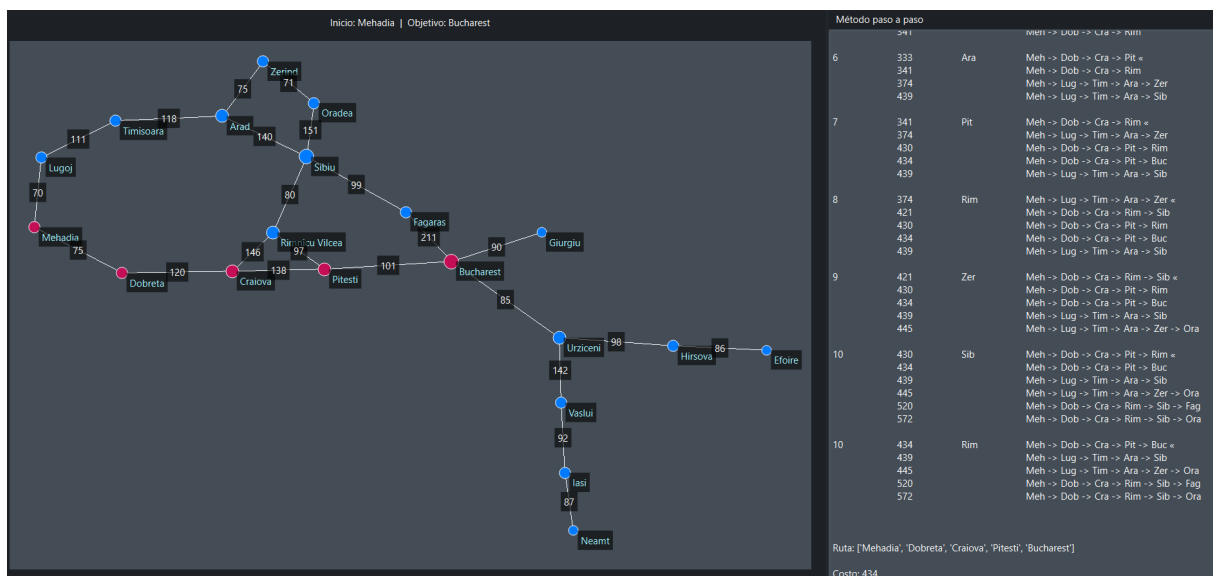


Figura 7: Ejemplo de Mejadia a Bucharest

Reinicio de la visualización: Para realizar una nueva ejecución, basta con seleccionar un nodo diferente o pulsar la herramienta **Limpiar** en la barra de herramientas.

5. Pseudocódigo del algoritmo Uniform Cost Search

Algorithm 1 Uniform_Cost_Search(grafo, inicio, objetivo)

```
1: cola ← lista con nodo inicial
2: visitados ← lista vacía
3: while cola no está vacía do
4:   ordenar(cola)
5:   datos ← sacar primer elemento de la cola
6:   if datos.actual == objetivo then
7:     return resultado
8:   end if
9:   if datos.actual no ha sido visitado then
10:    marcar datos.actual como visitado
11:    for all nodo en vecinos(datos.actual) do
12:      if nodo no ha sido visitado then
13:        peso ← obtenerPeso(datos.actual, nodo)
14:        nuevoCosto ← datos.peso_acumulado + peso
15:        nuevoCamino ← datos.camino + nodo
16:        agregar [nuevoCosto, nodo, nuevoCamino] a la cola
17:      end if
18:    end for
19:  end if
20: end while
21: return error
```

6. Formato del archivo JSON de entrada

El archivo JSON de entrada se utiliza para representar un grafo con nodos y enlaces, y consta de dos secciones principales: **nodes** y **links**.

Nodos (nodes)

La sección **nodes** es una lista de objetos, donde cada objeto representa un nodo (ciudad). Cada nodo debe incluir la clave **id** con un valor único el cual se mostrara en el el grafo:

```
"nodes": [
  {"id": "Arad"},
  {"id": "Bucharest"},
  {"id": "Craiova"},
  {"id": "Dobreta"},
  {"id": "Efoire"},
  ...
]
```

Enlaces (links)

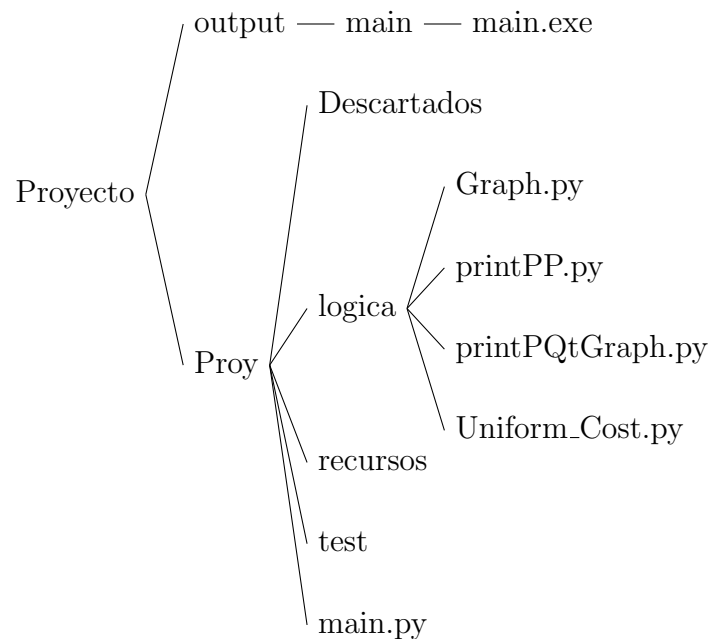
La sección `links` es una lista de objetos que describen las conexiones entre nodos. Cada objeto debe contener:

- `source`: nodo de origen.
- `target`: nodo de destino.
- `cost`: costo o peso de la arista.

```
"links": [  
    {"source": "Arad",          "target": "Sibiu",          "cost": 140},  
    {"source": "Arad",          "target": "Timisoara",        "cost": 118},  
    {"source": "Arad",          "target": "Zerind",           "cost": 75},  
    {"source": "Bucharest",      "target": "Fagaras",         "cost": 211},  
    ...  
]
```

7. Estructura del proyecto

Diagrama de carpetas



Descripción de carpetas y archivos

- **output/main/main.exe**: Ejecutable del programa.
- **Proy/Descartados**: Carpeta con opciones descartadas, pero viables para futuras versiones.

- **Proy/logica:** Contiene los módulos que proporcionan la funcionalidad principal del programa:
 - **Graph.py:** Crea la estructura del grafo.
 - **printPP.py:** Imprime en la terminal el paso a paso del algoritmo.
 - **printPQtGraph.py:** Genera el grafo utilizando PyQtGraph.
 - **Uniform_Cost.py:** Implementa el algoritmo de búsqueda de costo uniforme.
- **Proy/recursos:** Contiene íconos, el archivo JSON de ejemplo y otros recursos utilizados por la aplicación.
- **Proy/test:** Almacena pruebas y ejemplos para comprender el funcionamiento de las bibliotecas empleadas.
- **Proy/main.py:** Archivo principal que ejecuta la aplicación.